

CHAPTER 10

Advanced Register Data Flow Techniques

CHAPTER OUTLINE

- 10.1 Introduction
- 10.2 Value Locality and Redundant Execution
- 10.3 Exploiting Value Locality without Speculation
- 10.4 Exploiting Value Locality with Speculation
- 10.5 Summary

References

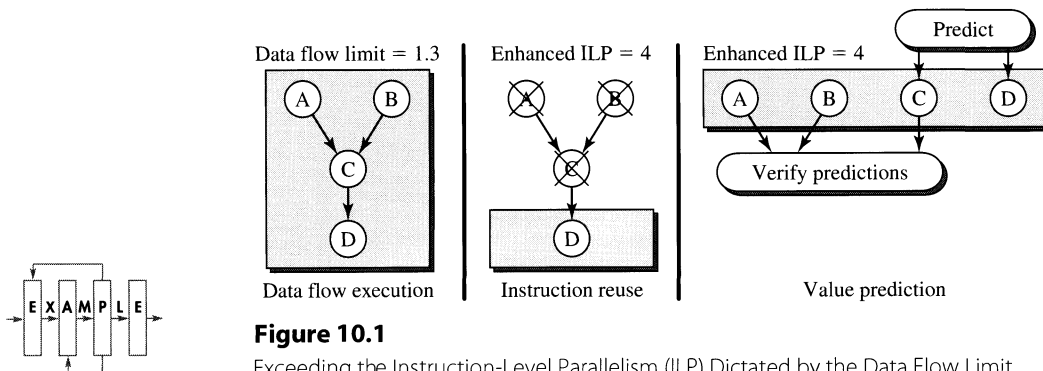
Homework Problems

10.1 Introduction

As we have learned, modern processors are fundamentally limited in performance by two program characteristics: control flow and data flow. The former was examined at length in our study of advanced instruction fetch techniques such as branch prediction, trace caches, and other high-bandwidth solutions to Flynn's bottleneck [Tjaden and Flynn, 1970]. Historically, these techniques have proved to be quite effective and many have been widely adopted in today's advanced processor designs. Nevertheless, resolving the limitations that control flow places on processor performance continues to be an extremely important area of research and advanced development. In Chapter 11, we will revisit this issue and focus on an active area of research that attempts to exploit multiple simultaneous flows of control to overcome bottlenecks caused by inaccuracies in branch prediction and inefficiencies in branch resolution. Before we do so, however, we will take a closer look at the performance limits that are caused by a program's data flow.

Earlier sections have already focused on resolving performance limitations caused by false or name dependences in a program. As the reader may recall, false dependences are caused by reuse of storage locations during program execution. Such reuse is induced by the fact that programmers and compilers must specify temporary operands with a finite number of unique register identifiers and are forced to reuse register identifiers once all available identifiers have been allocated. Furthermore, even if the instruction set provided the luxury of an unlimited number of registers and register identifiers, program loops induce reuse of storage identifiers, since multiple instances of a single static loop body can be in flight at the same time. Hence, false or name dependences are unavoidable. As we learned in Chapter 5, the underlying technique employed to resolve false dependences is to dynamically rename each destination operand to a unique storage location, and hence avoid unnecessary serialization of multiple writes to a shared location. This process of register renaming, first introduced as Tomasulo's algorithm in the IBM S/360-91 [Tomasulo, 1967] in the late 1960s, and detailed in Chapter 5, effectively removes false dependences and allows instructions to execute subject only to their true dependences. As has been the case with branch prediction, this technique has proved very effective, and various forms of register renaming have been implemented in numerous high-performance processor designs over the past four decades.

In this chapter, we turn our attention to techniques that attempt to elevate performance beyond what is achievable simply by eliminating false data dependences. A processor that executes instructions at a rate limited only by true data dependences is said to be operating at the *data flow limit*. Informally, a processor has achieved the data flow limit when each instruction in a program's dynamic data flow graph executes as soon as its source operands become available. Hence, an instruction's scheduled execution time is determined solely by its position in the data flow graph, where its position is defined as the longest path that leads to it in the data flow graph. For example, in Figure 10.1, instruction C is executed in cycle 2 because its true data dependences position it after instructions A and B, which execute in cycle 1. Recall that in a data flow graph



the nodes represent instructions, the edges represent data dependences between instructions, and the edges are weighted with the result latency of the producing instruction.

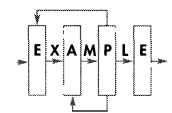
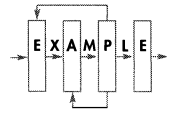
Given a data flow graph, we can compute a lower bound for a program's execution time by computing the height (i.e., the length of the longest existing path) of the data flow graph. The *data flow limit* represents this lower bound and, in turn, determines the maximum achievable rate of instruction execution (or ILP), which is defined as the number of instructions in the program divided by the height of the data flow graph. Just as an example, refer to the simple data flow graph shown on the left-hand side of Figure 10.1, where the maximum achievable ILP as determined by the data flow limit can be computed as 4 instructions

3 cycles of latency on the longest path through the graph = 1.3

In this chapter, we focus on two techniques—value prediction and instruction reuse—that exploit a program characteristic termed *value locality* to accelerate processing of instructions beyond the classic data flow limit. In this context, *value locality* describes the likelihood that a program instruction's computed result—or a similar, predictable result—will recur later during the program's continued execution. More broadly, the value locality of programs captures the empirical observation that a limited set of unique values constitute the majority of values produced and consumed by real programs. This property is analogous to the temporal and spatial locality that caches and memory hierarchies rely on, except that it describes the values themselves, rather than their storage locations.

The two techniques we consider exploit value locality by either nonspeculatively reusing the results of prior computation (in instruction reuse) or by speculatively predicting the results of future computation based on the results of prior executions (in value prediction). Both approaches allow a processor to obtain the results of an instruction earlier in time than its position in the data flow graph might indicate, and both are able to reduce the effective height of the graph, thereby increasing the rate of instruction execution beyond the data flow limit. For example, as shown in the middle of Figure 10.1, an instruction reuse scheme might recognize that instructions A, B, and C are repeating an earlier computation and could reuse the results of that earlier computation and allow instruction D to execute immediately, rather than having to wait for the results of A, B, and C. This would result in an effective throughput of four instructions per cycle. Similarly, the right side of Figure 10.1 shows how a data value prediction scheme could be used to enhance available instruction-level parallelism from a meager 1.3 instructions per cycle to an ideal 4 instructions per cycle by correctly predicting the results of instructions A, B, and C. Since A and B are predicted correctly, C need not wait for them to execute. Similarly, since C is correctly predicted, D need not wait for C to execute. Hence, all four instructions execute in parallel.

Figure 10.1 also illustrates a key distinction between instruction reuse and value prediction. In the middle case, invoking reuse completely avoids execution of instructions A, B, and C. In contrast, on the right, value prediction avoids the serializing effect of these instructions, but is not able to prevent their execution.



This distinction arises from a fundamental difference between the two techniques: instruction reuse guarantees value locality, while value prediction only predicts it. In the latter case, the processor must still verify the prediction by executing the predicted instructions and comparing their results to the predicted results. This is similar to branch prediction, where the outcome of the branch is predicted, almost always correctly, but the branch must still be executed to verify the correctness of the prediction. Of course, verification consumes execution bandwidth and requires a comparison mechanism for validating the results. Conversely, instruction reuse provides an a priori guarantee of correctness, so no verification code is needed. However, as we will find out in Section 10.3.2, this guarantee of correctness, while seemingly attractive, carries with it some baggage that can increase implementation cost and reduce the effectiveness of instruction reuse.

Neither value prediction nor instruction reuse, only relatively recently introduced in the literature, has yet been implemented in a real design. However, both demonstrate substantial potential for improving the performance of real programs, particularly programs where true data dependences—as opposed to structural or control dependences—place limits on achievable instruction-level parallelism. As with any new idea, there are substantial challenges involved in realizing that performance potential and reducing it to practice. We will explore some of these challenges and identify which have known realizable solutions and which require further investigation.

First, we will examine instruction reuse, since it has its roots in a historical and well-known program optimization called *memoization*. Memoization, which can be performed manually by the programmer, or automatically by the compiler, is a technique for short-circuiting complex computations by dynamically recording the outcomes of such computations. Subsequent instances of such computations then perform table lookups and reuse the results of prior computations whenever a new instance matches the same preconditions as an earlier instance. As may be evident to the reader, memoization is a nonspeculative technique, since it requires precisely correct preconditions to be satisfied before computation reuse is invoked. Similarly, instruction reuse is also nonspeculative and can be viewed as a hardware implementation of memoization at the instruction level.

Next, we will examine value prediction, which is fundamentally different due to its speculative nature. Rather than reusing prior executions of instructions, value prediction instead seeks to predict the outcome of a future instance of an instruction, based on prior outcomes. In this respect it is very similar to widely used history-based dynamic branch predictors (see Chapter 5), with one significant difference. While branch predictors collect outcome histories that can be quite deep (up to several dozen prior instances of branches can contribute their outcome history to the prediction of a future instance), the information content of the property they are predicting is very small, corresponding only to a single state bit that determines whether the branch is taken. In contrast, value predictors attempt to forecast full 32- or 64-bit values computed by register-writing instructions. Naturally, the challenges of accurately generating such predictions require much wider (full operand width) histories and additional mechanisms for avoiding mispredictions. Furthermore, generating predictions is only a small part of the implementation challenges required to

realize value prediction's performance potential. Just as with branch prediction, mechanisms for speculative execution based on predicted values as well as prediction verification and misprediction recovery, are all required for correct operation.

We begin with a discussion of value locality and its causes, and then consider many aspects of both nonspeculative techniques (e.g., instruction reuse) and speculative techniques (e.g., value prediction) for exploiting value locality. We examine all aspects of such techniques in detail; show how these techniques, though seemingly different, are actually closely related; and also describe how the two can be hybridized by combining elements of instruction reuse with an aggressive implementation of value prediction to reduce the cost of prediction verification.

10.2 Value Locality and Redundant Execution

In this section, we further explore the concept of *value locality*, which we define as the likelihood of a previously seen value recurring repeatedly within a storage location [Lipasti et al., 1996; Lipasti and Shen, 1996]. Although the concept is general and can be applied to any storage location within a computer system, here we consider the value locality of general-purpose or floating-point registers immediately following instructions that write those registers. A plethora of previous work on dynamic branch prediction has focused on an even more restricted application of value locality, namely, the prediction of a single condition bit based on its past behavior. Many of the ideas in this chapter can be viewed as a logical continuation of that body of work, extending the prediction of a single bit to the prediction of an entire 32- or 64-bit register.

10.2.1 Causes of Value Locality

Intuitively, it seems that it would be a very difficult task to discover any useful amount of value locality in a register. After all, a 32-bit register can contain any one of over four billion values—how could one possibly predict which of those is even somewhat likely to occur next? As it turns out, if we narrow the scope of our prediction mechanism by considering each static instruction individually, the task becomes much easier, and we are able to accurately predict a significant fraction of values being written to the register file.

What is it that makes these values predictable? After examining a number of real-world programs, we have found that value locality exists primarily because real-world programs, run-time environments, and operating systems are *general by design*. That is, not only are they implemented to handle contingencies, exceptional conditions, and erroneous inputs, all of which occur relatively rarely in real life, but they are also often designed with future expansion and code reuse in mind. Even code that is aggressively optimized by modern, state-of-the-art compilers exhibits these tendencies. The following empirical observations result from our examination of many real programs, and they should help the reader understand why value locality exists:

- *Data redundancy.* Frequently, the input sets for real-world programs contain data that have little variation. Examples of this are sparse matrices that contain many zeros, text files with white space, and empty cells in spreadsheets.

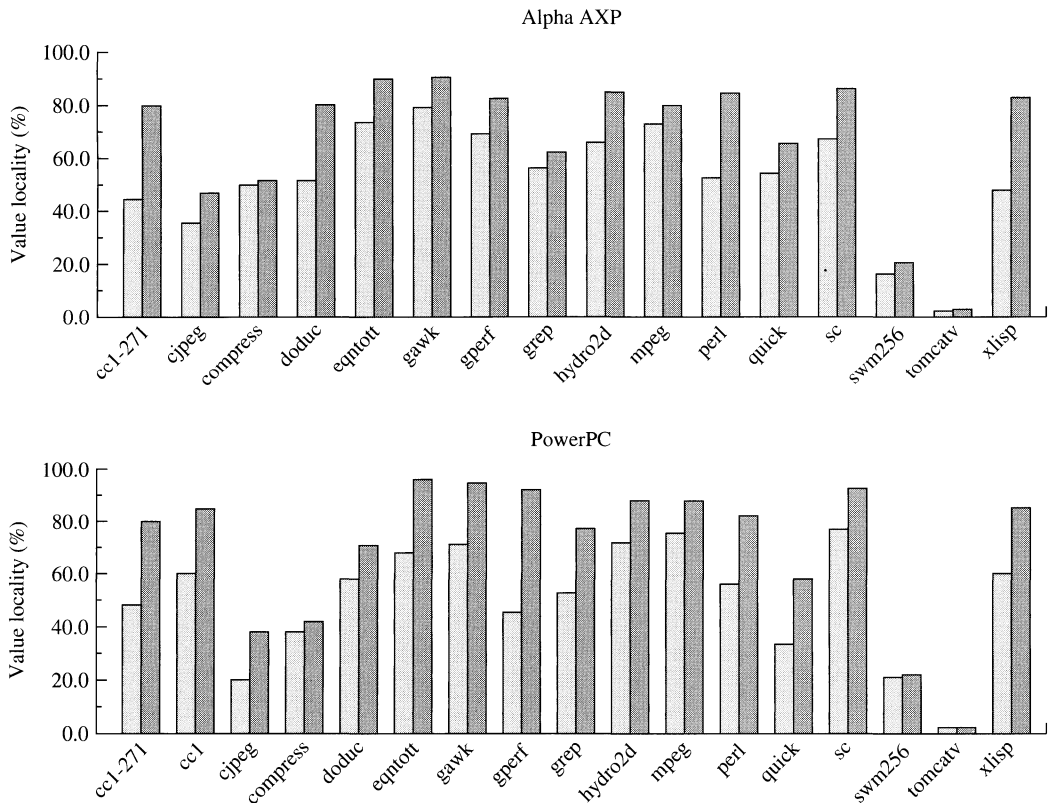
- *Error checking.* Checks for infrequently occurring conditions often compile into loads of what are effectively run-time constants.
- *Program constants.* It is often more efficient to generate code to load program constants from memory than code to construct them with immediate operands.
- *Computed branches.* To compute a branch destination, say for a switch statement, the compiler must generate code to load a register with the base address for the branch jump table, which is often a run-time constant.
- *Virtual function calls.* To call a virtual function, the compiler must generate code to load a function pointer, which can often be a run-time constant.
- *Glue code.* Because of addressability concerns and linkage conventions, the compiler must often generate glue code for calling from one compilation unit to another. This code frequently contains loads of instruction and data addresses that remain constant throughout the execution of a program.
- *Addressability.* To gain addressability to nonautomatic storage, the compiler must load pointers from a table that is not initialized until the program is loaded, and thereafter remains constant.
- *Call-subgraph identities.* Functions or procedures tend to be called by a fixed, often small, set of functions, and likewise tend to call a fixed, often small, set of functions. Hence, the calls that occur dynamically often form identities in the call graph for the program. As a result, loads that restore the link register as well as other callee-saved registers can have high value locality.
- *Memory alias resolution.* The compiler must be conservative about stores that may alias with loads, and will frequently generate what appear to be redundant loads to resolve those aliases. These loads are likely to exhibit high degrees of value locality.
- *Register spill code.* When a compiler runs out of registers, variables that may remain constant are spilled to memory and reloaded repeatedly.
- *Convergent algorithms.* Often, value locality is caused by algorithms that the programmer chose to implement. One common example is convergent algorithms, which iterate over a data set until global convergence is reached; quite often, local convergence will occur before global convergence, resulting in redundant computation in the converged areas.
- *Polling algorithms.* Another example of how algorithmic choices can induce value locality is the use of polling algorithms instead of more efficient event-driven algorithms. In a polling algorithm, the most likely outcome is that the event being polled for has not yet occurred, resulting in redundant computation to repeatedly check for the event.

Naturally, many of these observations are subject to the particulars of the instruction set, compiler, and run-time environment being employed, and one could argue that some could be eliminated with changes to the ISA, compiler, or run-time

environment, or by applying aggressive link-time or run-time code optimizations. However, such changes and improvements have been slow to appear; the aggregate effect of the listed (and other) factors on value locality is measurable and significant today on the two modern RISC instruction sets that we examined, both of which provide state-of-the-art compilers and run-time systems. It is worth pointing out, however, that the value locality of particular static loads in a program can be significantly affected by compiler optimizations such as loop unrolling, loop peeling, and tail replication, since these types of transformations tend to create multiple instances of a load that may now exclusively target memory locations with high or low value locality.

10.2.2 Quantifying Value Locality

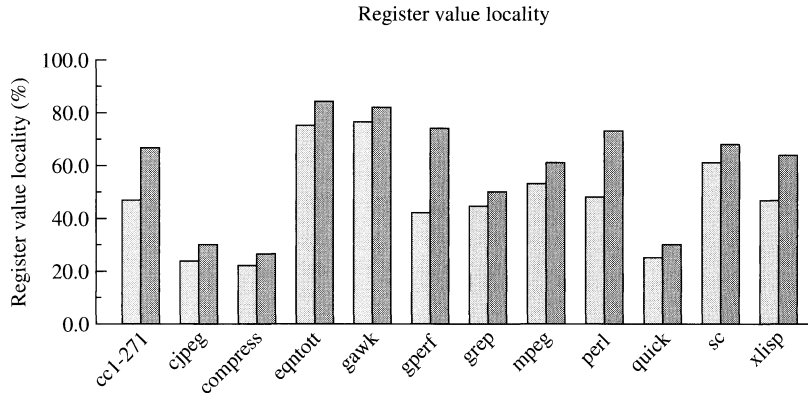
Figure 10.2 shows the value locality for load instructions in a variety of benchmark programs. The value locality for each benchmark is measured by counting the number of times each static load instruction retrieves a value from memory



The light bars show value locality for a history depth of one, while the dark bars show it for a history depth of sixteen.

Figure 10.2

Load Value Locality.



The light bars show value locality for a history depth of one, while the dark bars show it for a history depth of four.

Figure 10.3

Register Value Locality.

that matches a previously seen value for that static load, and dividing by the total number of dynamic loads in the benchmark. Two sets of numbers are shown, one (light bars) for a history depth of 1 (i.e., check for matches against only the most recently retrieved value), while the second set (dark bars) has a history depth of 16 (i.e., check against the last 16 unique values). We see that even with a history depth of 1, most of the integer programs exhibit load value locality in the 50% range, while extending the history depth to 16 (along with a hypothetical perfect mechanism for choosing the right one of the 16 values) can improve that to better than 80%. What this means is that the vast majority of static loads exhibit very little variation in the values that they load during the course of a program's execution. Unfortunately, three of these benchmarks (*cjpeg*, *swm256*, and *tomcatv*) demonstrate poor load value locality.

Figure 10.3 shows the *average value locality* for all instructions that write an integer or floating-point register in each of the benchmarks. The value locality of each static instruction is measured by counting the number of times that instruction writes a value that matches a previously seen value for that static instruction and dividing by the total number of dynamic occurrences of that instruction. The average value locality of a benchmark is the dynamically weighted average of the value localities of all the static instructions in that benchmark. Two sets of numbers are shown, one (light bars) for a history depth of one (i.e., we check for matches against only the most recently written value), while the second set (dark bars) has a history depth of four (i.e., we check against the last four unique values). We see that even with a history depth of one, most of the programs exhibit value locality in the 40% to 50% range (average 51%), while extending the history depth to four (along with a perfect mechanism for choosing the right one of the four values) can improve that to the 60% to 70% range (average 66%). What that means is that a majority of static instructions exhibit very little variation in the

values that they write into registers during the course of a program's execution. Once again, three of these benchmarks—*cjpeg*, *compress*, and *quick*—demonstrate poor register value locality.

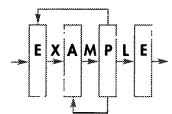
In summary, all the programs studied here, and many others studied exhaustively elsewhere, demonstrate significant amounts of value locality, for both load instructions and all register-writing instructions [Lipasti et al., 1996; Lipasti and Shen, 1996; 1997; Mendelson and Gabbay, 1997; Gabbay and Mendelson, 1997; 1998a; 1998b; Sazeides and Smith, 1997; Calder et al., 1997; 1999; Wang and Franklin, 1997; Burtscher and Zorn, 1999; Sazeides, 1999]. This property has been independently verified for at least a half-dozen different instruction sets and compilers and a large number of workloads including both user-state and kernel-state execution.

10.3 Exploiting Value Locality without Speculation

The widespread occurrence of value locality in real programs creates opportunities for increasing processor performance. As we have already outlined, both speculative and nonspeculative techniques are possible. We will first describe nonspeculative techniques for exploiting value locality, since related techniques have been known for a long time. A recent proposal has reinvigorated interest in such techniques by advocating *instruction reuse* [Sodani and Sohi, 1997; 1998; Sodani, 2000], which is a pure hardware technique for reusing the result of a prior execution of an instruction. In its simplest form, an instruction reuse mechanism avoids the structural and data hazards caused by execution of an instruction whenever it discovers an identical instruction execution within its history mechanism. In such cases, it simply reuses the historical outcome saved in the instruction reuse buffer and discards the fetched instruction without executing it. Dependent instructions are able to issue and execute immediately, since the result is available right away. Because of value locality, such reuse is often possible since many static instructions repeatedly compute the same result.

10.3.1 Memoization

Instruction reuse has its roots in a historical and well-known program optimization called *memoization*. Memoization, which can be performed manually by the programmer or automatically by the compiler, is a technique for short-circuiting complex computations by dynamically recording the outcomes of such computations and reusing those outcomes whenever possible. For example, each <operand, result> pair resulting from calls to the function *fibonacci(x)* shown in Figure 10.4 can be recorded in a memoization table. Subsequent instances of such computations then perform table lookups and reuse the results of prior computations whenever a new instance matches the same preconditions as an earlier instance. Continuing our example, a memoized version of the *fibonacci(x)* function checks to see if the current call matches an earlier call, and then returns the value of the earlier call immediately, rather than executing the full routine to recompute the Fibonacci series sum.



```

/* fibonacci series computation */
int fibonacci(x) {
    int result = 0;
    if (x==0)
        result = 0;
    else if (x<3)
        result = 1;
    else {
        result = fibonacci(x-2);
        result += fibonacci(x-1);
    }
    return result;
}
/* memoized version */
int memoized_fibonacci(x) {
    if (seen_before(x))
        return memoized_result(x);
    else {
        int result = fibonacci(x);
        memoize(x,result);
        return result;
    }
}

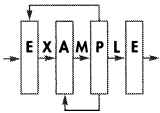
/* linked list example */
int ordered_linked_list_insert(record *x) {
    int position=0;
    record *c,*p;
    c=head;
    while (c && (c->data < x->data)) {
        ++position;
        p = c;
        c = c->next;
    }
    if (p) {
        x->next = p->next;
        p->next = x;
    } else
        head = x;
    return position;
}

```

The call to *fibonacci(x)*, shown on the left, can easily be memoized, as shown in the *memoized_fibonacci(x)* function. The call to *ordered_linked_list(record *x)* would be very difficult to memoize due to its reliance on global variables and side effect updates to those global variables.

Figure 10.4

Memoization Example.



Besides the overhead of recording and checking for memoized results, the main shortcoming of memoization is that any computation that is memoized must be guaranteed to be free of side effects. That is, the computation must not itself modify any global state, nor can it rely on external modifications to the global state. Rather, all its inputs must be clearly specified so the memoization table lookup can verify that they match the earlier instance; and all its outputs, or effects on the rest of the program, must also be clearly specified so the reuse mechanism can perform them correctly. Again, in our simple *fibonacci(x)* example, the only input is the operand *x*, and the only output is the Fibonacci series sum corresponding to *x*, making this an excellent candidate for memoization. On the other hand, a procedure such as *ordered_linked_list_insert(record *x)*, also shown in Figure 10.4, would be a poor candidate for memoization, since it both depends on the global state (a global *head* pointer for the linked list as well as the nodes in the linked list) and modifies the global state by updating the *next* pointer of a linked list element. Correct memoization of this type of function would require checking that the head pointer and none of the elements of the list had changed since the previous invocation.

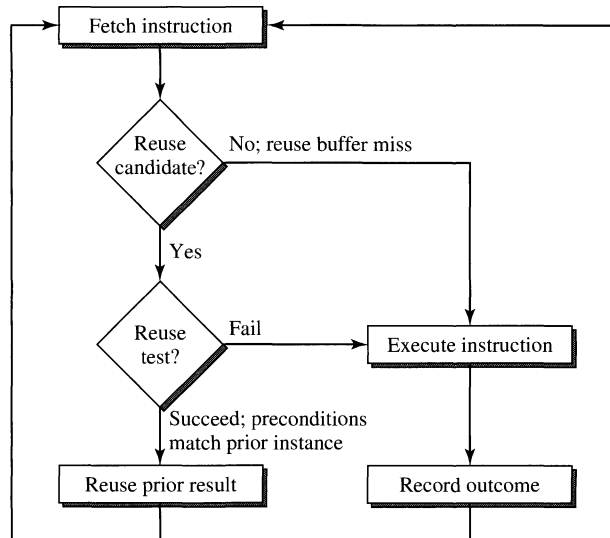
Nevertheless, memoization is a powerful programming technique that is widely deployed and can be very effective. Clearly, memoization is a nonspeculative technique, since it requires precisely correct preconditions to be satisfied before reuse is invoked.

10.3.2 Instruction Reuse

Conceptually, instruction reuse is nothing more than a hardware implementation of memoization at the instruction level. It exposes additional instruction-level parallelism by decoupling the execution of a consumer instruction from its producers whenever it finds that the producers need not be executed. This is possible whenever the reuse mechanism finds that a producer instruction matches an earlier instance in the reuse history and is able to safely reuse the results of that prior instance. Sodani and Sohi's initial proposal for instruction reuse advocated reuse of an individual machine instruction whenever the operands to that instruction were shown to be invariant with respect to a prior instance of that instruction [Sodani and Sohi, 1997]. A more advanced mechanism for recording and reusing sequences of data-dependent instructions was also described. This mechanism stored the data dependence relationships between instructions in the reuse history table and could automatically reuse a data flow region of instructions (i.e., a subgraph of the dynamic data flow graph) whenever all the inputs to that region were shown to be invariant. Subsequent proposals have also considered expanding the reuse scope to include basic blocks as well as instruction traces fetched from a trace cache (refer to Chapter 5 for more details on how trace caches operate).

All these proposals for reuse share the same basic approach: the execution of an individual instruction or set of instructions is recorded in a history structure that stores the result of the computation for later reuse. The set of instructions can be defined by either control flow (as in basic block reuse and trace reuse) or data flow (as in data flow region reuse). The history structure must have a mechanism that guarantees that its contents remain coherent with subsequent program execution. Finally, the history structure has a lookup mechanism that allows subsequent instances to be checked against the stored instances. A hit or match during this lookup triggers the reuse mechanism, which allows the processor to skip execution of the reuse candidates. As a result, the processor eliminates the structural and data dependences caused by the reuse candidates and is able to fast-forward to subsequent program instructions. This process is summarized in Figure 10.5.

10.3.2.1 The Reuse History Mechanism. Any implementation of reuse must have a mechanism for remembering, or memoizing, prior executions of instructions or sequences of instructions. This history mechanism must associate a set of preconditions with a previously computed result. These preconditions must exactly specify both the computation to be performed as well as all the *live inputs*, or operands that can affect the outcome of the computation. For instruction reuse, the computation to be performed is specified by a program counter (PC) tag that uniquely identifies a static instruction in the processor's address space, while the live inputs are both register and memory operands to that static instruction. For block reuse, the computation is specified by the address range of the instructions in the basic block, while the live inputs are all the source register and memory operands that are live on entry to the basic block. For trace reuse, the computation corresponds to a trace cache entry, which is uniquely identified by the fetch address and a set of conditional branch



After an instruction is fetched, the history mechanism is checked to see whether the instruction is a candidate for reuse. If so, and if the instructions preconditions match the historical instance, the historical instance is reused and the fetched instruction is discarded. Otherwise, the instruction is executed as always, and its outcome is recorded in the history mechanism.

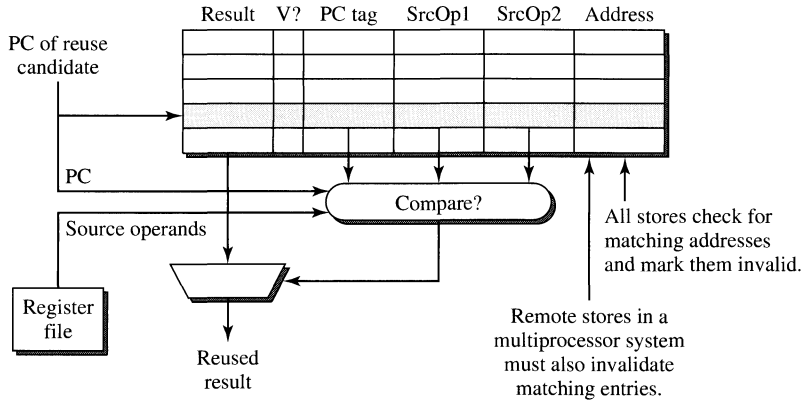
Figure 10.5

Instruction Reuse.

outcomes that specify the control flow path of the trace. By extension, all operands that are live on entry to the trace must also be specified.

The key attribute of the preconditions stored in the reuse buffer is that they uniquely specify the set of events that led to the computation of the memoized result. Hence, if that precise set of events ever occurs again, the computation need not be performed again. Instead, the memoized result can be substituted for the result of the repeated computation. However, just as with the memoization example in Figure 10.4, care must be taken that the preconditions in fact fully specify all the events that might affect the outcome of the computation. Otherwise, the reuse mechanism may introduce errors into program execution.

Indexing and Updating the Reuse Buffer. The history mechanism, or *reuse buffer*, is illustrated in Figure 10.6. It is usually indexed by low-order bits of the PC, and it can be organized as a direct-mapped, set-associative, or fully associative structure. Additional index information can be provided by including input operand and value bits in the index and/or the tag; such an approach enables multiple instances of the same static instruction, but with varying input operands, to coexist in the reuse buffer. The reuse buffer is updated dynamically, as instructions or groups of instructions retire from the execution window; this may require a multi-ported or heavily banked structure to accommodate high throughput. There are



The instruction reuse buffer stores all the preconditions required to guarantee correct reuse of prior instances of instructions. For ALU and branch instructions, this includes a PC tag and source operand values. For loads and stores, the memory address must also be stored, so that intervening writes to that address will invalidate matching reuse entries.

Figure 10.6

Instruction Reuse Buffer.

also the usual design space issues regarding replacement policy and writeback policy (for multilevel history structures), similar to design issues for caches and cache hierarchies.

Reuse Buffer Organization. The reuse buffer can be organized to store history for individual instructions (i.e., each entry corresponds to a single instruction), for basic blocks, for traces (effectively integrating reuse history in the trace cache), or for data flow regions. There are scalability issues related to tracking live inputs for large numbers of instructions per reuse entry. For example, a basic block history mechanism may have to store up to a dozen or more live inputs and half as many results, given a basic block size of six or more instructions, each with two source operands and one destination. Similar scalability problems exist for proposed trace reuse mechanisms, which attempt to reuse entire traces of up to 16 instructions. Imagine increasing the width of the one-instruction-wide structure shown in Figure 10.6 to accommodate 16 instances of all the columns. Clearly, building such wide structures and wide comparators for checking reuse preconditions presents a challenging task.

Specifying Live Inputs. Live register inputs to a reuse entry can be specified either by *name* or by *value*. Specifying by name means recording either the architected register number for a register operand or the address for a memory operand. Specifying by value means recording the actual value of the operand instead of its name. Either way, all live inputs must be specified to maintain correctness, since failure to specify a live input can lead to incorrect reuse, where a computation is reused even though a subtle change to an unrecorded live input

could cause a different result to occur. Sodani and Sohi investigated mechanisms that specified register operands both by name and by value, but only considered specifying memory operands by name. The example reuse buffer in Figure 10.6 specifies register source operands by value and memory locations by name.

Validating Live Inputs. To validate the live inputs of a reuse candidate, one must verify that the inputs stored in the reuse entry match the current architected values of those operands; this process is called the *reuse test*. Unless all live inputs are validated, reuse must not occur, since the reused result may not be correct. For named operands, this property is guaranteed by a coherence mechanism (explained next) that checks all program writes against the reuse buffer. For operands specified by value, the reuse mechanism must compare the current architected values against those in the reuse entry to check for a match. For register operands, this involves reading the current values from the architected register file and comparing them to the values stored in the reuse entry. Note that this creates considerable additional demand for read ports into the physical register file, since all operands for all reuse candidates must be read simultaneously. For memory operands specified by value, performing the reuse test would involve fetching the operand values from memory in order to compare them. Clearly, there is little to be gained here, since fetching the operands from memory in order to compare them is no less work than performing the memory operation itself. Hence, all reuse proposals to date specify memory operands by name, rather than by value. In Figure 10.6, each reuse candidate must fetch its source operands from the register file and compare them with the values stored in the reuse buffer.

Reuse Buffer Coherence Mechanism. To guarantee correctness, the reuse buffer must remain coherent with program execution that occurs between insertion of an entry into the reuse buffer and any subsequent reuse of that entry. To remain coherent, any intervening writes to either registers or memory that conflict with named live inputs must be properly reflected in the reuse buffer. The coherence mechanism is responsible for tracking all writes performed by the program (or other programs running on other processors in a multiprocessor system) and making sure that any named live inputs that correspond to those writes are marked invalid in the reuse structure. This prevents invalid reuse from occurring in cases where a named live input has changed. If live inputs are specified by value, rather than by name, intervening writes need not be detected, since the live input validation will compare the resulting architected and historic values and will trigger reuse only when the values match. Note that for named inputs, the coherence mechanism must perform an associative lookup over all the live inputs in the reuse buffer for every program write. For long names (say, 32- or 64-bit memory addresses), this associative lookup can be prohibitively expensive even for modest history table sizes. In Figure 10.6, all stores executed by the processor must check for matching entries in the reuse buffer and must invalidate the entry if its address matches the store. Similarly, in a multiprocessor system, all remote writes must invalidate matching entries in the reuse buffer.

As a final note, in systems that allow self-modifying code, the coherence mechanism must also track writes to the instruction addresses that are stored in the reuse buffer and must invalidate any matching reuse entries. Failure to do so could result in the reuse of an entry that no longer corresponds to the current program image. Similarly, the semantics of instructions that are used to invalidate instruction cache entries (e.g., *icbi* in the PowerPC architecture) must be extended to also invalidate reuse buffer entries with matching tags.

10.3.2.2 Reuse Mechanism. Finally, to gain performance benefit from reuse, the processor must be able to eliminate or reduce data and structural dependences for reused instructions by omitting the execution of these instructions and skipping ahead to subsequent work. This seems straightforward, but may require nontrivial modifications to the processor's data and control paths. First, reuse candidates (whether individual instructions or groups of instructions) must inject their results into the processor's architected state; since the data paths for doing so in real processors often only allow functional units to write results into the register file, this will probably involve adding write ports to an already heavily multiported physical register file. Second, instruction wakeup and scheduling logic will have to be modified to accommodate reused instructions with effectively zero cycles of result latency. Third, the reuse candidates must enter the processor's reorder buffer in order to maintain support for precise exceptions, but must simultaneously bypass the issue queues or reservation stations; this nonstandard behavior will introduce additional control path complexity. Finally, reused memory instructions must still be tracked in the processor's load/store queue (LSQ) to maintain correct memory reference ordering. Since LSQ entries are typically updated after instruction issue based on addresses generated during execution, this may also entail additional data paths and LSQ write ports that allow updates to occur from an earlier (prior to issue or execute) pipeline stage.

In summary, implementing instruction reuse will require substantial redesign or modification of existing control and data paths in a modern microprocessor design. This requirement may be the reason that reuse has not yet appeared in any real designs; the changes are substantial enough that they are likely to be incorporated only into a brand-new, clean-slate design.

10.3.3 Basic Block and Trace Reuse

Subsequent proposals have extended Sodani and Sohi's original proposal for instruction reuse to encompass sets of instructions defined by control flow [Huang and Lilja, 1999; Gonzalez et al., 1999]. In these proposals, similar mechanisms for storing and looking up reuse history are employed, but at the granularity of basic blocks or instruction traces. In both cases, the control flow unit (either basic block or trace) is treated as an atomically reusable computation. In other words, partial reuse due to partial matching of input operands is disallowed. Expanding the scope of instruction reuse to basic blocks and traces increases the potential benefit per reuse instance, since a substantial chunk of instructions can be directly bypassed. However, it also decreases the likelihood of finding a matching reuse entry, since the

likelihood that a set of a half-dozen or dozen live inputs are identical to a previous computation is much lower than the likelihood of finding individual instructions within those groups that can be reused. Also, as discussed earlier, there are scalability issues related to conducting a reuse test for the large numbers of live inputs that basic blocks and traces can have. Only time will tell if reuse at a coarser control-flow granularity will prove to be more effective than instruction-level reuse.

10.3.4 Data Flow Region Reuse

In contrast to subsequent approaches that attempt to reuse groups of instructions based on control flow, Sodani also proposed an approach for storing and reusing data flow regions of instructions (the S_{n+d} and S_{v+d} schemes). This approach requires a bookkeeping scheme that embeds pointers in the reuse buffer to connect data-dependent instructions. These pointers can then be traversed to reuse entire subgraphs of the data flow graph; this is possible since the reuse property is transitive with respect to the data flow graph. More formally, any instruction whose data flow antecedents are all reuse candidates (i.e., they all satisfy the reuse test) is also a reuse candidate. By applying this principle inductively, a reusable data flow region can be constructed, resulting in a set of connected instructions that are all reusable. The reusable region is constructed dynamically by following the data dependence pointers embedded in the reuse table. Dependent instructions are connected by these edges, and any successful reuse test results are propagated along these edges to dependent instructions. The reuse test for the dependent instructions simply involves checking that all live input operands originate in instructions that were just reused or otherwise pass the reuse test. If this condition is satisfied, meaning that all operands are found to be invariant or to originate from reused antecedents, the dependent instructions themselves can be reused. The reuse test can be performed either by name (in the S_{n+d} scheme) or by value (in the S_{v+d} scheme).

Maintaining the integrity of the data dependence pointers presents a difficult challenge in a dynamically managed structure: Whenever an entry in the reuse buffer is replaced, all pointers to that entry become stale. All these stale pointers must be found and removed to prevent subsequent accesses to the reuse buffer from resulting in incorrect transitive propagation of reusability. Sodani proposed an associative lookup mechanism that automatically invalidates all such pointers on every replacement. Clearly, the expense and complexity of associative lookup coupled with frequent replacement prevent this from being a scalable solution. Alternative schemes that store dependence pointers in a separate, smaller structure which can feasibly support associative lookup are also possible, though unexplored in the current literature.

Subsequent work by Connors and Hwu [1999] proposes implementing region-level reuse strictly in software by modifying the compiler to generate code that performs the reuse test for data flow regions constructed by the compiler. This approach checks the live input operands and invokes region reuse by omitting execution of the region and immediately writing its results to the architected state whenever a matching history entry is found. In fact, this work takes us full circle

back to software-based memoization techniques and establishes that automated, profile-driven techniques for memoization are indeed feasible and desirable.

10.3.5 Concluding Remarks

In summary, various schemes for reuse of prior computation have been proposed. These proposals are conceptually similar to the well-understood technique of memoization and vary primarily in the granularity of reuse and details of implementation. They all rely on the program characteristic of value locality, since without it, the likelihood of identifying reuse candidates would be very low. Reuse techniques have not been adopted in any real designs to date; yet they show significant performance potential if all the implementation challenges can be successfully overcome.

10.4 Exploiting Value Locality with Speculation

Having considered nonspeculative techniques for exploiting value locality and enhancing instruction-level parallelism, we now address speculative techniques for doing the same. Before delving into the details of value prediction, we step back to consider a theoretical basis for speculative execution—the *weak dependence model* [Lipasti and Shen, 1997; Lipasti, 1997].

10.4.1 The Weak Dependence Model

As we have learned in our study of techniques for removing false dependences, the implied inter-instruction precedences of a sequential program are an over-specification and need not be rigorously enforced to meet the requirements of the sequential execution model. The actual program semantics and inter-instruction dependences are specified by the control flow graph (CFG) and the data flow graph (DFG). As long as the serialization constraints imposed by the CFG and the DFG are not violated, the execution of instructions can be overlapped and reordered to achieve better performance by avoiding the enforcement of implied but unnecessary precedences. This can be achieved by Tomasulo’s algorithm or more recent, modern reorder-buffer-based implementations. However, true inter-instruction dependences must still be enforced. To date, all machines enforce such dependences in a rigorous fashion that involves the following two requirements:

- Dependences are determined in an absolute and exact way; that is, two instructions are identified as either dependent or independent, and when in doubt, dependences are pessimistically assumed to exist.
- Dependences are enforced throughout instruction execution; that is, the dependences are never allowed to be violated, and are enforced continuously while the instructions are in flight.

Such a traditional and conservative approach for program execution can be described as adhering to the *strong dependence model*. The traditional strong

dependence model is overly rigorous and unnecessarily restricts available parallelism. An alternative model that enables aggressive techniques such as value prediction is the *weak dependence model*, which specifies that:

- Dependences need not be determined exactly or assumed pessimistically, but instead can be optimistically approximated or even temporarily ignored.
- Dependences can be temporarily violated during instruction execution as long as recovery can be performed prior to affecting the permanent machine state.

The advantage of adopting the weak dependence model is that the program semantics as specified by the CFG and DFG need not be completely determined before the machine can process instructions. Furthermore, the machine can now speculate aggressively and temporarily violate the dependences as long as corrective measures are in place to recover from misspeculation. If a significant percentage of the speculations are correct, the machine can effectively exceed the performance limit imposed by the traditional strong dependence model.

Conceptually speaking, a machine that exploits the weak dependence model has two interacting engines. The front-end engine assumes the weak dependence model and is highly speculative. It tries to make predictions about instructions in order to aggressively process instructions. When the predictions are correct, these speculative instructions effectively will have skipped over or *folded out* certain pipeline stages. The back-end engine still uses the strong dependence model to validate the speculations, to recover from misspeculation, and to provide history and guidance information to the speculative engine. In combining these two interacting engines, an unprecedented level of instruction-level parallelism can be harvested without violating the program semantics. The edges in the DFG that represent inter-instruction dependences are now enforced in the critical path only when misspeculations occur. Essentially, these dependence edges have become probabilistic and the serialization penalties incurred due to enforcing these dependences are eliminated or masked whenever correct speculations occur. Hence, the traditional *data flow limit* based on the length of the critical path in the DFG is no longer a hard limit that cannot be exceeded.

10.4.2 Value Prediction

We learned in Section 10.2.2 that the register writes in many programs demonstrate a significant degree of value locality. This discovery opens up exciting new possibilities for the microarchitect. Since the results of many instructions can be accurately predicted before they are issued or executed, dependent instructions are no longer bound by the serialization constraints imposed by operand data flow. Instructions can now be scheduled speculatively with additional degrees of freedom to better utilize existing functional units and hardware buffers and are frequently able to complete execution sooner since the critical paths through dependence graphs have been collapsed. However, in order to exploit value locality and reap all these benefits, a variety of hardware mechanisms must be implemented: one for accurately predicting values (the *value prediction unit*); microarchitectural support for executing

with speculative values; a mechanism for verifying value predictions; and finally a recovery mechanism for restoring correctness in cases where incorrectly predicted values were introduced into the program's execution.

10.4.3 The Value Prediction Unit

The value prediction unit is responsible for generating accurate predictions for speculative consumption by the processor core. The two competing factors that determine the efficacy of the value prediction unit are *accuracy* and *coverage*; a third factor related to coverage is the predictor's *scope*. Accuracy measures the predictor's ability to avoid mispredictions, while coverage measures the predictor's ability to predict as many instruction outcomes as possible. A predictor's scope describes the set of instructions that the predictor targets. Achieving high accuracy (e.g., few mispredictions) generally implies trading off some coverage, since any scheme that eliminates mispredictions will likely also eliminate some correct predictions. Conversely, achieving high coverage will likely reduce accuracy for the same reason: Aggressively pursuing every prediction opportunity is likely to result in a larger number of mispredictions.

Grasping the tradeoff between accuracy and coverage is easy if you consider the two extreme cases. At one extreme, a predictor can achieve 100% coverage by indiscriminately predicting all instructions; this will result in poor accuracy, since many instructions are inherently unpredictable and will be mispredicted. At the other extreme, a predictor can achieve 100% accuracy by not predicting any instructions and eliminating all mispredictions; of course, this will result in 0% coverage since none of the predictable instructions will be predicted either. The designer's challenge is to find a point between these two extremes that provides both high accuracy and high coverage.

Limiting the *scope* of the value predictor to focus on a particular class of instructions (e.g., load instructions) or some other dynamically or statically determined subset can make it easier to improve accuracy and/or coverage for that subset, particularly with a fixed implementation cost budget.

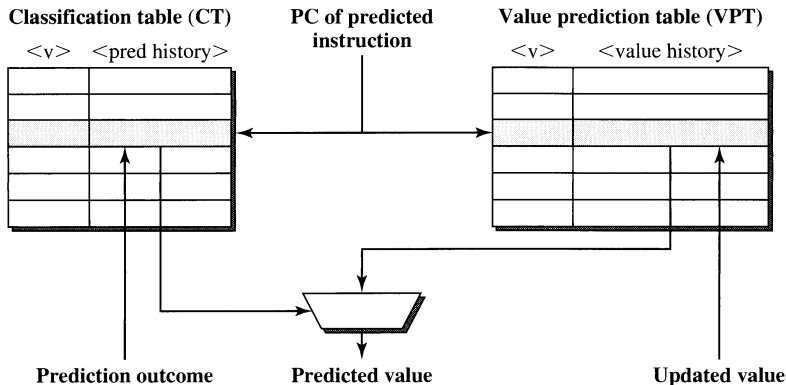
Building a value prediction unit that achieves the right balance of accuracy and coverage requires careful tradeoff analysis that must consider the performance effects of variations in coverage (i.e., proportional variation in freedom for scheduling of instructions for execution and changes in the height of the dynamic data flow graph) and variations in accuracy (i.e., fewer or more frequent mispredictions). This analysis will vary depending on minute structural and timing details of the microarchitecture being considered and requires detailed register-transfer-level simulation for correct tradeoff analysis. The analysis is further complicated by the fact that greater coverage does not always result in better performance, since only a relatively small subset of predictions are actually critical for performance. Similarly, improved accuracy may not improve performance either, since the mispredictions that were eliminated may also not have been critical for performance. A recent study by Fields, Rubin, and Bodik [2001] quantitatively demonstrates this by directly measuring the critical path of a program's execution and showing that relatively few correct value predictions actually remove edges along the critical

path. They suggest limiting the value predictor's scope to only those instructions that are on the critical (i.e., longest) path in the program's data flow graph.

10.4.3.1 Prediction Accuracy. A naive value prediction scheme would simply endorse all possible predictions generated by the prediction scheme and supply them as speculative operands to the execution core. However, as published reports have shown, value predictors vary dramatically in their accuracy, at times providing as few as 18% correct predictions. Clearly, naive consumption of incorrect predictions is not only intellectually unsatisfying; it can lead to performance problems due to misprediction penalties. While it is theoretically possible to implement misprediction recovery schemes that have no direct performance penalty, practical difficulties will likely preclude such schemes (we discuss one possible approach in Section 10.4.4.5 under the heading Data Flow Eager Execution). Hence, beginning with the initial proposal for value prediction, researchers have described confidence estimation techniques for improving predictor accuracy.

Confidence Estimation. Confidence estimation techniques associate a confidence level with each value prediction, and they are used to filter incorrect predictions to improve predictor accuracy. If a prediction exceeds some confidence threshold, the processor core will actually consume the predicted value. If it does not, the predicted value is ignored and execution proceeds nonspeculatively, forcing the dependent operations to wait for the producer to finish computing its result. Typically, confidence levels are established with a history mechanism that increments a counter for every correct prediction and decrements or resets the counter for every incorrect prediction. Usually, there is a counter associated with every entry in the value prediction unit, although multiple counters per entry and multiple entries per counter have also been studied. The classification table shown in Figure 10.7 is a simple example of a confidence estimation mechanism. The design space for confidence estimators has been explored quite extensively in the literature to date and is quite similar to the design space for dynamic branch predictors (as discussed in Chapter 5). Design parameters include the choice of single or multiple levels of history; indexing with prediction outcome history, PC value, or some hashed combination; the number of states and transition functions in the predictor entry state machines; and so on. Even a relatively simple confidence estimation scheme, such as the one described in Figure 10.7, can provide prediction accuracy that eliminates more than 90% of all mispredictions while sacrificing less than 10% of coverage.

10.4.3.2 Prediction Coverage. The second factor that measures the efficacy of a value prediction unit is prediction coverage. The simple value predictors that were initially proposed simply remembered the previous value produced by a particular static instruction. An example of such a *last value predictor* is shown in Figure 10.7. Every time an instruction executes, the *value prediction table* (VPT) is updated with its result. As part of the update, the confidence level in the classification table is incremented if the prior value matched the actual outcome, and decremented otherwise. The next time the same static instruction is fetched, the previous value is



The internal structure of a simple value prediction unit (VPU). The VPU consists of two tables: the *classification table* (CT) and the *value prediction table* (VPT), both of which are direct-mapped and indexed by the instruction address (PC) of the instruction being predicted. Entries in the CT contain two fields: the *valid* field, which consists of either a single bit that indicates a valid entry or a partial or complete tag field that is matched against the upper bits of the PC to indicate a valid field; and the *prediction history*, which is a saturating counter of 1 or more bits. The prediction history is incremented or decremented whenever a prediction is correct or incorrect, respectively, and is used to classify instructions as either predictable or unpredictable. This classification is used to decide whether or not the result of a particular instruction should be predicted. Increasing the number of bits in the saturating counter adds hysteresis to the classification process and can help avoid erroneous classifications by ignoring anomalous values and/or destructive interference.

Figure 10.7

Value Prediction Unit.

retrieved along with the current confidence level. If the confidence level exceeds a fixed threshold, the predicted value is used; otherwise, it is discarded.

Simple last value predictors provide roughly 40% coverage over a set of general-purpose programs. Better coverage can be obtained with more sophisticated predictors that either provide additional context to allow the predictor to choose from multiple prior values (*history-based predictors*) or are able to detect predictable sequences and compute future, previously unseen, values (*computational predictors*).

History-Based Predictors. The simplest history-based predictors remember the most recent value written by a particular static instruction and predict that the same value will be computed by the next dynamic instance of that instruction. More sophisticated predictors provide a means for storing multiple different values for each static instruction, and then use some scheme to choose one of those values as the predicted one. For example, the *last- n value predictor* proposed by Burtcher and Zorn [1999] uses a scheme of prediction outcome histories to choose one of n values stored in the value prediction table. Alternatively, the *finite-context-method* (FCM) predictor proposed by Sazeides and Smith [1997] also stores multiple values, but chooses one based on a finite context of recent values observed during program execution, rather than strictly by PC value. This value context is analogous to the branch outcome context captured by a branch history register that is

used successfully to implement two-level branch predictors. The FCM scheme is able to capture periodic sequences of values, such as the set of pointer addresses loaded by the traversal of a linked list. The FCM predictor has been shown to reach prediction coverage in excess of 90% for certain workloads, albeit with considerable implementation cost for storing multiple values and their contexts.

Computational Predictors. Computational predictors attempt to capture a predictable pattern in the sequence of values generated by a static instruction and then compute the next instance in the sequence. They are fundamentally different from history-based predictors since they are able to generate predicted values that have not occurred in prior program execution. Gabbay and Mendelson [1997] first proposed a *stride predictor* that detects a fixed stride in the value sequence and is able to compute the next value by adding the observed stride to the prior value. A stride predictor requires additional hardware: to detect strides it must use a 32- or 64-bit subtraction unit to extract the stride and a comparator to check the extracted stride against the previous stride instance; it needs additional space in the value prediction table to store the stride value and some additional confidence estimation bits to indicate a valid stride; and, finally, it needs an adder to add the prior value to the stride to compute each new prediction. Stride prediction can be quite effective for certain workloads; however, it is not clear if the additional storage, arithmetic hardware, and complexity are justified.

More advanced computational predictors have been discussed, but none have been formally proposed to date. Clearly, there is a continuum in the design space for computational predictors between the two extremes of history-based prediction with no computational ability and full-blown preexecution, where all the architected state is made available as *context* to the predictor, and which simply anticipates the semantics of the actual program to precompute its results. While the latter extreme is obviously neither practical nor useful, since it simply replicates the functionality of the processor's execution core, the interesting question that remains is whether there is a useful middle ground where at least a subset of program computation can be abstracted to the point that a computational predictor of reasonable cost is able to replicate it with high accuracy. Clearly, sophisticated branch predictors are able to abstract 95% or more of many programs' control flow behavior; whether sophisticated computational value predictors can ever reach the same goal for a program's data flow remains an open question.

Hybrid Predictors. Finally, analogous to the hybrid or combining branch predictors described in Chapter 9, various schemes that combine multiple heterogeneous predictors into a single whole have been proposed. Such a hybrid prediction scheme might combine a last value predictor, a stride predictor, and a finite-context predictor in an attempt to reap the benefits of each. Hybrid predictors can enable not only better overall coverage, but can also allow more efficient and smaller implementations of advanced prediction schemes, since they can be targeted only to the subset of static instructions that require them. A very effective hybrid predictor was proposed by Wang and Franklin [1997].

Implementation Issues. Several studies have examined various implementation issues for value prediction units. These issues encompass the size, organization, accessibility, and sensitivity to update latency of value prediction structures, and they can be difficult to solve, particularly for complex computational and hybrid predictors. In general, solutions such as clever hash functions for indexing the tables and banking the structure to enable multiple simultaneous accesses have been shown to work well. A recent proposal that shifts complex value predictor access to completion time, and stores the results of that access in a simple, direct-mapped table or directly in a trace cache entry, is able to shift much of the access complexity away from the timing-critical front end of the processor pipeline [Lee and Yew, 2001]. Another intriguing proposal refrains from storing values in a separate history structure by instead predicting that the needed value is already in the register file, and storing a pointer to the appropriate register [Tullsen and Seng, 1999]. Surprisingly, this approach works reasonably well, especially if the compiler allocates register names with some knowledge of the values stored in the registers.

10.4.3.3 Prediction Scope. The final factor determining the efficacy of a value prediction unit is its intended prediction scope. The initial proposal for value prediction focused strictly on load instructions, limiting its scope to a subset of instructions generally perceived to be critical for performance. Reducing load latency by predicting and speculatively consuming the values returned by those loads has been shown to improve performance and reduce the effect of structural hazards for highly contended cache ports, and should increase memory-level parallelism by allowing loads that would normally be blocked by a data flow–antecedent cache miss to execute in parallel with the miss.

The majority of proposed prediction schemes target all register-writing instructions. However, there are some interesting exceptions. Sodani and Sohi [1998] point out that register contents that are directly used to resolve conditional branches should probably not be predicted, since such value predictions are usually less accurate than the tailored predictions made by today’s sophisticated branch predictors. This issue was sidestepped in the initial value prediction work, which used the PowerPC instruction set architecture, in which all conditional branches are resolved using dedicated condition registers. Since only general-purpose registers were predicted, the detrimental effect of value mispredictions misguidedly overriding correct branch predictions was kept to a minimum. In instruction sets similar to MIPS or PISA (used in Sodani’s work), there are no condition registers, so a scheme that value predicts all general-purpose registers will also predict branch source operands and can directly and adversely affect branch resolution.

Several researchers have proposed focusing value predictions on only those data dependences that are deemed critical for performance [Calder et al., 1999]. This has several benefits: The extra work of useless predictions can be avoided; predictors with better accuracy and coverage and lower implementation cost can be devised; and mispredictions that occur for useless predictions can be reduced or eliminated. Fields, Rubin, and Bodik [2001] demonstrate many of these benefits in

their recent proposal for deriving data dependence criticality by a novel approach to monitoring out-of-order instruction execution.

10.4.4 Speculative Execution Using Predicted Values

Just as with instruction reuse, value prediction requires microarchitectural support for taking advantage of the early availability of instruction results. However, there is a fundamental difference in the required support due to the speculative nature of value prediction. Since instruction reuse is preceded by a reuse test that guarantees its correctness, the microarchitectural changes outlined in Section 10.3.2.2 consist primarily of additional bandwidth into the bookkeeping structures within an out-of-order superscalar processor. In contrast, value prediction—an inherently speculative technique—requires more pervasive support in the microarchitecture to handle detection of and recovery from misspeculation. Hence, value prediction implies microarchitectural support for *value-speculative execution*, for *verifying predictions*, and for *misprediction recovery*. We will first describe a minimal approach for supporting value-speculative execution; then we will discuss more advanced verification and recovery strategies.

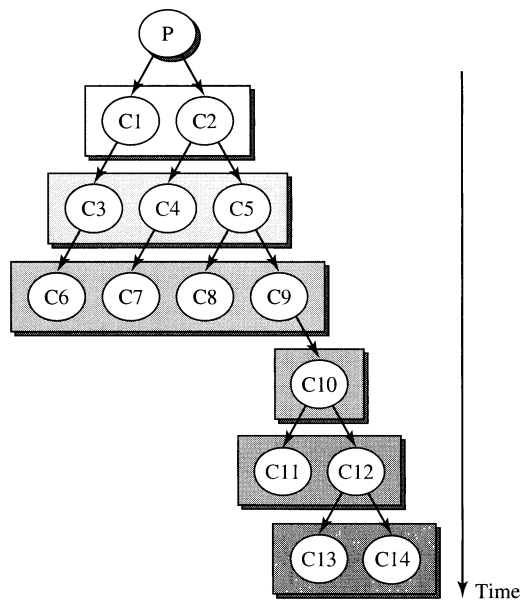
10.4.4.1 Straightforward Value Speculation. At first glance, it seems that speculative execution using predicted values maps quite naturally onto the structures that a modern out-of-order superscalar processor already provides. First of all, to support value speculation, we need a mechanism for storing and forwarding predictions from the value prediction unit to the dependent instructions: the existing rename buffers or rename registers serve this purpose quite well. Second, we need a mechanism to issue dependent instructions speculatively; the standard out-of-order issue logic, with minor modifications, will work for this purpose as well. Third, we need a mechanism for detecting mispredicted values. The obvious solution is to augment the reservation stations to hold the predicted output values for each instruction, and provide additional data paths from the reservation station and the functional unit output to a comparator that checks these values for equality and signals a misprediction when the comparison fails. Finally, we need a way to recover from mispredictions. If we treat value mispredictions the same way we treat branch mispredictions, we can simply recycle the branch misprediction recovery mechanism that flushes out speculative instructions and refetches all instructions following the mispredicted one. Surprisingly, these minimal modifications are sufficient for correctness in a uniprocessor system,¹ and can even provide nontrivial speedup as long as the predictor is highly accurate and mispredictions are relatively rare. However, more sophisticated verification and recovery techniques can lead to higher-performance designs, but require additional complexity. We discuss such techniques in the following.

¹A recent publication discusses why they are not sufficient in a cache-coherent multiprocessor: essentially, value prediction removes the natural reference ordering between data-dependent loads by allowing a dependent load to execute before a preceding load that computes its address; multiprocessor programs that rely on such dependence ordering for correctness can fail with the naive value prediction scheme described here. The interested reader is referred to Martin et al. [2001] for further details.

10.4.4.2 Prediction Verification. Prediction verification is analogous to the reuse test that guarantees correctness for instruction reuse. In other words, it must guarantee that the predicted outcome of a value-predicted instruction matches the actual outcome, as determined by the architected state and the semantics of the instruction. The most straightforward approach to verification is to execute the predicted instruction and then compare the outcome of the execution with the value prediction. Naively, this implies appending an ALU-width comparator to each functional unit to verify predictions. Since the latency through a comparator is equivalent to the delay through an ALU, most proposals have assumed an extra cycle of latency to determine whether or not a misprediction occurred.

Prediction verification serves two purposes. The first is to trigger a recovery action whenever a misprediction occurs; possible recovery actions are discussed in Section 10.4.4.4. The second purpose is more subtle and occurs when there is no misprediction: The fact that a correct prediction was verified may now need to be communicated to dependent instructions that have executed speculatively using the prediction. Depending on the recovery model, such speculatively executed instructions may continue to occupy resources within the processor window until they are found to be nonspeculative. For example, in a conventional out-of-order microprocessor, instructions can only enter the issue queues or reservation stations in program order. Once they have issued and executed, there is no data or control path that enables placing them back in the issue queue to reissue. In such a microarchitecture, an instruction that consumed a predicted source operand and issued speculatively would need to remain in the issue queue or reservation station in case it needed to reissue with a future corrected operand. Since issue queue slots are an important and performance-critical hardware resource, timely notification of the fact that an instruction's input operands were not mispredicted can be important for reducing structural hazards.

As mentioned, the most straightforward approach for misprediction detection is to wait until a predicted instruction's operands are available before executing the instruction and comparing its result with its predicted result. The problem with this approach is that the instruction's operands themselves may be speculative (that is, the producer instructions may have been value predicted, or, more subtly, some data flow antecedent of the producer instructions may have been value predicted). Since speculative input operands beget speculative outputs, a single predicted value can propagate transitively through a data flow graph for a distance limited only by the size of the processor's instruction window, creating a wavefront of speculative operand values (see Figure 10.8). If a speculative operand turns out to be incorrect, verifying an instruction's own prediction with that incorrect operand may cause the verification to succeed when it should not or to fail when it should succeed. Neither of these is a correctness issue; the former case will be caught since the incorrect input operand will eventually be detected when the misprediction that caused it is verified, while the latter case will only cause unnecessary invocations of the recovery mechanism. However, for this very reason, the latter can cause a performance problem, since correctly executed instructions are reexecuted unnecessarily.



The speculative operand wavefront traverses the dynamic data flow graph as a result of the predicted outcome of instruction P. Its consumers C1 and C2 propagate the speculative property to their consumers C3, C4, and C5, and so on. Serial propagation of prediction verification status propagates through the data flow graph in a similar manner. Parallel propagation, which requires a tag broadcast mechanism, allows all speculatively executed dependent instructions to be notified of verification status in a single cycle.

Figure 10.8

The Speculative Operand Wavefront.

Speculative Verification. A similar problem arises when speculative operands are used to resolve branch instructions. In this scenario, a correctly predicted branch can be resolved incorrectly due to an incorrect value prediction, resulting in a branch misprediction redirect. The straightforward solution to these two problems is to disallow prediction verification (whether value or branch) with speculative inputs. The shortcoming of this solution is that performance opportunity is lost whenever a correct speculative input would have appropriately resolved a mispredicted branch or corrected a value misprediction. There is no definitive answer as to the importance of this performance effect; however, the recent trend toward deep execution pipelines that are very performance-sensitive to branch mispredictions would lead one to believe that any implementation decision that delays the resolution of incorrectly predicted branches is the wrong one.

Propagating Verification Results. As an additional complication, in order to delay verification until all input operands are nonspeculative, there must be a mechanism in place that informs the instruction whether its input operands have been verified. In its simplest form, such a mechanism is simply the reorder buffer (ROB); once an instruction becomes the oldest in the ROB, it can infer that all its

data flow antecedents are verified, so it can now also be verified. However, delaying verification until an instruction is next to commit has negative performance implications, particularly for mispredicted conditional branch instructions. Hence, a mechanism that propagates verification status of operands through the data flow graph is desirable. Two fundamental design alternatives exist: The verification status can be propagated serially, along the data dependence edges, as instructions are verified; or it can be broadcast in parallel. Serial propagation can be piggybacked on the existing broadcast result used to wake up dependent instructions in out-of-order execution. Parallel broadcast is more expensive, and it implies tagging operand values with all speculative data flow antecedents, and then broadcasting these tags as the predictions are verified. Parallel broadcast has a significant latency benefit, since entire dependence chains can become nonspeculative in the cycle following verification of some long-latency instruction (e.g., cache miss) at the head of the chain. As discussed, this instantaneous commit can reduce structural hazards by freeing up issue queue or reservation station slots right away, instead of waiting for serial propagation through the data flow graph.

10.4.4.3 Data Flow Region Verification. One interesting opportunity for improving the efficiency of value prediction verification arises from the concept of data flow regions. Recall that data flow regions are subgraphs of the data flow graph that are defined by the set of instructions that are reachable from a set of live inputs. As proposed by Sodani and Sohi [1997], a data flow region can be reused en masse if the set of live inputs to the region meets the reuse test. The same property can also be exploited to verify the correctness of all the value predictions that occur in a data flow region. A mechanism similar to the one described in Section 10.3.4 can be integrated into the value prediction unit to construct data flow regions by storing data dependence pointers in the value prediction table. Subsequent invocation of value predictions from a self-consistent data flow region then leads to a reduction in verification scope. Namely, as long as the data flow region mechanism guarantees that all the predictions within the region are consistent with each other, only the initial predictions that correspond to the live inputs to the data flow region need to be verified. Once these initial predictions are verified via conventional means, the entire data flow region is known to be verified, and the remaining instructions in the region need not ever be executed or verified.

This approach is strikingly similar to data flow region reuse, and it requires quite similar mechanisms in the value prediction table to construct data flow region information and guarantee its consistency (these issues are discussed in greater detail in Section 10.3.4). However, there is one fundamental difference: data flow region reuse requires the live inputs to the data flow region to be either unperturbed (if the reuse test is performed by name) or unchanged and available in the register file (if the reuse test is performed by value). Integrating data flow regions with value prediction, however, avoids these limitations by deferring the reuse test indefinitely, until the live inputs are available within the processor's execution window. Once the live inputs have all been verified, the entire data flow region can be notified of its nonspeculative status and can retire without ever executing. This

should significantly reduce structural dependences and contention for functional units for programs where reusable data flow regions make up a significant portion of the instructions executed.

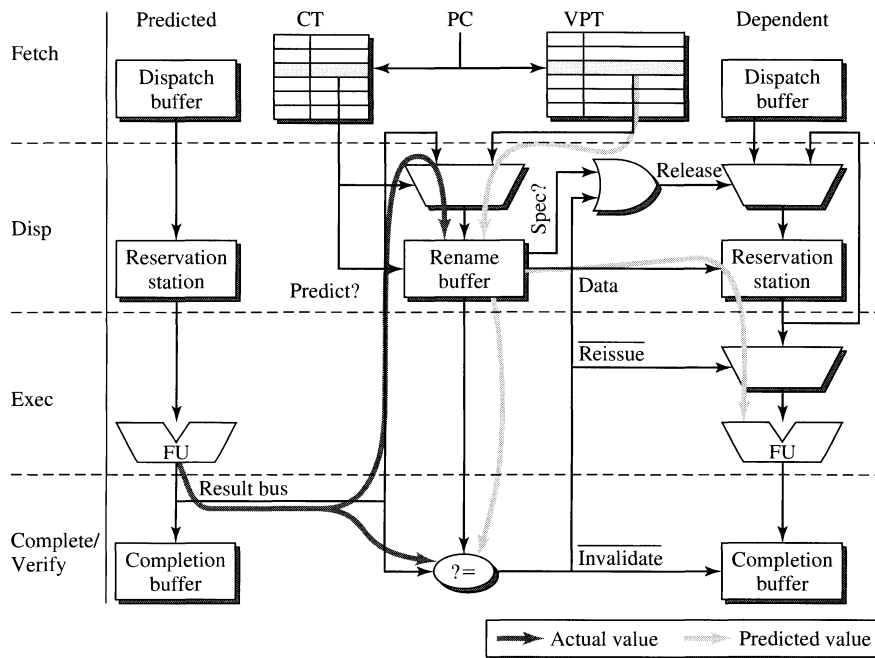
10.4.4.4 Misprediction Recovery via Refetch. There are two approaches to recovering from value mispredictions: *refetch* and *selective reissue*. As already mentioned, refetch-based recovery builds on the branch misprediction recovery mechanism which is present in almost every modern superscalar processor. In this approach, value mispredictions are treated exactly as branch mispredictions: All instructions that follow the mispredicted instruction in program order are flushed out of the processor, and instruction fetch is redirected to refetch these instructions. The architected state is restored to the instruction boundary following the mispredicted instruction, and the refetched instructions are guaranteed to not be polluted by any mispredicted values, since such mispredicted values do not survive the refetch.

The most attractive feature of refetch-based misprediction recovery is that it requires very few changes to the processor, assuming the mechanism is already in place for redirecting mispredicted branches. On the other hand, it has the obvious drawback that the misprediction penalty is quite severe. Studies have shown that in a processor with a refetch policy for recovering from value mispredictions, highly accurate value prediction is a requirement for gaining performance benefit. Without highly accurate value prediction—usually brought about by a high-threshold confidence mechanism—performance can in fact degrade due to the excessive refetches. Unfortunately, a high-threshold confidence mechanism also inevitably reduces prediction coverage, resulting in a processor design that fails to capture all the potential performance benefit of value prediction.

10.4.4.5 Misprediction Recovery via Selective Reissue. Selective reissue provides a potential solution to the performance limitations of refetch-based recovery. With selective reissue, only those instructions that are data dependent on a mispredicted value are required to reissue. Implementing selective reissue requires a mechanism for propagating misprediction information through the data flow graph to all dependent instructions. Just as was the case for propagating verification information, reissue information can also be propagated serially or in parallel. A serial mechanism can easily piggyback on the existing result bus that is used to wake up dependent instructions in an out-of-order processor. With serial propagation, the delay for communicating a reissue condition is proportional to the data flow distance from the misprediction to the instruction that must reissue. It is conceivable, although unlikely, that the reissue message will never catch up with the speculative operand wavefront illustrated in Figure 10.8, since both propagate through the data flow graph at the same rate of one level per cycle. Furthermore, even if the reissue message does eventually reach the speculative operand wavefront, the serial propagation delay of the reissue message can cause excessive wasted execution along the speculative operand wavefront. Hence, researchers have also proposed broadcast-based mechanisms that communicate reissue commands in parallel to all dependent instructions.

In such a parallel mechanism, speculative value-predicted operands are provided with a unique tag, and all dependent instructions that execute with such operands must propagate those tags to their dependent instructions. On a misprediction, the tag corresponding to the mispredicted operand is broadcast so that all data flow descendants realize they must reissue and reexecute with a new operand. Figure 10.9 illustrates a possible implementation of value prediction with parallel-broadcast selective reissue.

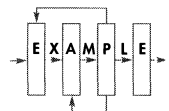
Misprediction Penalty with Selective Reissue. With refetch-based misprediction recovery, the misprediction penalty is comparable to a branch misprediction penalty and can run to a dozen or more cycles in recent processors with very deep



The dependent instruction shown on the right uses the predicted result of the instruction on the left, and is able to issue and execute in the same cycle. The VP Unit predicts the values during fetch and dispatch, then forwards them speculatively to subsequent dependent instructions via a rename buffer. The dependent instruction is able to issue and execute immediately, but is prevented from completing architecturally and retains possession of its reservation station until its inputs are no longer speculative. Speculatively forwarded values are tagged with the uncommitted register writes they depend on, and these tags are propagated to the results of any subsequent dependent instructions. Meanwhile, the predicted instruction executes on the right, and the predicted value is verified by a comparison against the actual value. Once a prediction is verified, its tag is broadcast to all active instructions, and all the dependent instructions can either release their reservation stations and proceed into the completion unit (in the case of a correct prediction), or restart execution with the correct register values (if the prediction was incorrect).

Figure 10.9

Example of Value Prediction with Selective Reissue.



pipelines. The goal of selective reissue is to mitigate this penalty by reducing the number of cycles that elapse between determining that a misprediction occurred and correctly re-executing data-dependent instructions. Assuming a single additional cycle for prediction verification, the apparent best case would be a single cycle of misprediction penalty. That is to say, the dependent instruction executes one cycle later than it would have had there been no value prediction.

The penalty occurs only when a dependent instruction has already executed speculatively but is waiting in its reservation station for one of its predicted inputs to be verified. Since the value comparison takes an extra cycle beyond the pipeline result latency, the dependent instruction will reissue and execute with the correct value one cycle later than it would have had there been no prediction. In addition, the earlier incorrect speculative issue may have caused a structural hazard that prevented other useful instructions from dispatching or executing. In those cases where the dependent instruction has not yet executed (due to structural or other unresolved data dependences), there is no penalty, since the dependent instruction can issue as soon as the actual computed value is available, in parallel with the value comparison that verifies the prediction.

Data Flow Eager Execution. It is possible to reduce the misprediction penalty to zero cycles by employing *data flow eager execution*. In such a scheme, the dependent instruction is speculatively re-executed as soon as the nonspeculative operand becomes available. In other words, there is a second shadow issue of the dependent instruction as if there had been no earlier speculative one. In parallel with this second issue, the prediction is verified, and in case of correct prediction, the shadow issue is squashed. Otherwise, the shadow issue is allowed to continue, and execution continues as if the value prediction had never occurred, with effectively zero cycles of misprediction penalty. Of course, the data flow eager shadow issue of all instructions that depend on value predictions consumes significant additional execution resources, potentially overwhelming the available functional units and slowing down computation. However, given a wide machine with sufficient execution resources, this may be a viable alternative for reducing the misprediction penalty. Prediction confidence could also be used to gate data flow eager execution. In cases where prediction confidence is high, eager execution is disabled; in cases where confidence is low, eager execution can be used to mitigate the misprediction penalty.

The Effect of Scheduling Latency. In a canonical out-of-order processor that implements the modern equivalent of Tomasulo's algorithm, instruction scheduling decisions are made in a single cycle immediately preceding the actual execution of the instructions that are selected for execution. Such a scheme allows the scheduler to react immediately to dynamic events, such as detection of store-to-load aliases or cache misses, and issue alternative, independent instructions in subsequent cycles. However, cycle time constraints have led recent designs to abandon this property, resulting in instruction schedulers that create an execution schedule several cycles in advance of the actual execution. This effect, called the *scheduling latency*, inhibits the scheduler's ability to react to dynamic events. Of course, value misprediction

detection is a dynamic event, and the fact that several modern processor designs (e.g., Alpha 21264 and Intel Pentium 4) have multicycle scheduling latency will necessarily increase the value misprediction penalty on such machines. In short, the value misprediction penalty is in fact the sum of the scheduling latency and the verification latency. Hence, a processor with three-cycle scheduling latency and a one-cycle verification latency would have a value misprediction latency of four cycles. However, even in such designs it is possible to reduce the misprediction penalty via *data flow eager execution*. Of course, the likelihood that execution resources will be overwhelmed by this approach increases with scheduling latency, since the number of eagerly executed and squashed instructions is proportional to this latency.

10.4.4.6 Further Implications of Selective Reissue

Memory Data Dependences. Selective reissue requires all data-dependent instructions to reissue following a value misprediction. While it is fairly straightforward to identify register data dependences and reissue dependent instructions, memory data dependences can cause a subtle problem. Namely, memory data dependences are defined by register values themselves; if the register values prove to be incorrect due to a value misprediction, memory data dependence information may need to be reconstructed to guarantee correctness and to determine which additional instructions are in fact dependent on the value misprediction. For example, if the mispredicted value was used either directly or indirectly to compute an address for a load or store instruction, the load/store queue or any other alias resolution mechanism within the processor may have incorrectly concluded that the load or store is or is not aliased with some other store or load within the processor's instruction window. In such cases, care must be taken to ensure that memory dependence information is recomputed for the load or store whose address was polluted by the value misprediction. Alternatively, the processor can disallow the use of value-predicted operands in address generation for loads or stores. Of course, doing so will severely limit the ability of value prediction to improve memory-level parallelism. Note that this problem does not occur with a refetch recovery policy, since memory dependence information is explicitly recomputed for all instructions following the value misprediction.

Changes to Scheduling Logic. Reissuing instructions requires nontrivial changes to the scheduling logic of a conventional processor. In normal operation, instructions issue only one time, once their input operands become available and a functional unit is available. However, with selective reissue, an instruction may have to issue multiple times, once with speculative operands and again with corrected operands. All practical out-of-order implementations partition the active instruction window into two disjoint sets: instructions waiting to issue (these are the instructions still in reservation stations or issue queues), and instructions that have issued but are waiting to retire. This partitioning is driven by cycle-time demands that limit the total number of instructions that can be considered for issue in a single cycle. Since instructions that have already issued need not be considered for reissue, they are moved out of reservation stations or issue queues into the second partition (instructions waiting to retire).

Unfortunately, with selective reissue, a clean partition is no longer possible, since instructions that issued with speculative operands may need to reissue, and hence should not leave the issue queue or reservation station. There are two solutions to this problem: either remove speculatively issued instructions from the reservation stations, but provide an additional mechanism to reinsert them if they need to reissue; or keep them in the reservation stations until their input operands are no longer speculative. The former solution introduces significant additional complexity into the front-end control and data paths and must also deal with a possible deadlock scenario. One such scenario occurs when all reservation station entries are full of newer instructions that are data dependent on an older instruction that needs to be reinserted into a reservation station so that it can reissue. Since there are no reservation stations available, and none ever become available since all the newer instructions are waiting for the older instruction, the older instruction cannot make forward progress, and can never retire, leading to a deadlocked system. Note that refetch-based recovery does not have this problem, since all newer data-dependent instructions are flushed out of the reservation stations upon misprediction recovery.

Hence, the latter solution of forcing speculatively issued instructions to retain their reservation station entries is proposed most often. Of course, this approach requires a mechanism for promoting speculative operands to nonspeculative status. A parallel or serial mechanism like the ones described in Section 10.4.4.2 will suffice for this purpose. In addition to the complexity introduced by having to track the verification status of operands, this solution has the additional slight problem that it increases the occupancy of the reservation station entries. Without value prediction, a dependent instruction releases its reservation station in the same cycle that it issues, which is the cycle following computation of its last input operand. With the proposed scheme, even though the instruction may have issued much earlier with a value-predicted operand, the reservation station itself is occupied for one additional cycle beyond operand availability, since the entry is not released until after the predicted operand is verified, one cycle later than it is computed.

Existing Support for Data Speculation. Note that existing processors that do not implement value prediction, but do support other forms of data speculation (for example, speculating that a load is not aliased to a prior store), may already support a limited form of selective reissue. The Intel Pentium 4 is one such processor and implements selective reissue to recover from cache hit speculation; here, data from a cache access are forwarded to dependent instructions before the tag match that validates a cache hit has completed. If there is a tag mismatch, the dependent instructions are selectively reissued. If this kind of selective reissue scheme already exists, it can also be used to support value misprediction recovery. However, the likelihood of being able to reuse an existing mechanism is reduced by the fact that existing mechanisms for selective reissue are often tailored for speculative conditions that are resolved within a small number of cycles (e.g., tag mismatch or alias resolution). The fact that the speculation window extends only for a few cycles allows the speculative operand wavefront (see Figure 10.8) to propagate

through only a few levels in the data flow graph, which in turn limits the total number of instructions that can issue speculatively. If the selective reissue mechanism exploits this property and is somehow restricted to handling only a small number of dependent operations, it is not useful for value prediction, since the speculation window for value prediction can extend to tens or hundreds of cycles (e.g., when a load that misses the cache is value predicted) and can encompass the processor's entire instruction window. However, the converse does hold: If a processor implements selective reissue to support value prediction, the same mechanism can be reused to support recovery for other forms of data speculation.

10.4.5 Performance of Value Prediction

Numerous published studies have examined the performance potential of value prediction. The results have varied widely, with reported performance effects ranging from minor slowdowns to speedups of 100% or more. Achievable performance depends heavily on many of the factors already mentioned, including particular details of the machine model and pipeline structure, as well as workload choice. Some of the factors affecting performance are

- The degree of value locality present in the programs or workloads.
- The dynamic dependence distance between correctly predicted instructions and the instructions that consume their results. If the compiler has already scheduled dependent instructions to be far apart, reducing result latency with value prediction may not provide much benefit.
- The instruction fetch rate achieved by the machine model. If the fetch rate is fast relative to the pipeline's execution rate, value prediction can significantly improve execution throughput. However, if the pipeline is fetch-limited, value prediction will not help much.
- The coverage achieved by the value prediction unit. Clearly, the more instructions are predicted, the more performance benefit is possible. Conversely, poor coverage results in limited opportunity.
- The accuracy of the value prediction unit. Achieving a high ratio between correct and incorrect predictions is critical for reaping significant performance benefit, since mispredictions can slow the processor down.
- The misprediction penalty of the pipeline implementation. As discussed, both the recovery policy (refetch versus reissue) and the efficiency of the recovery policy can severely affect the performance impact of mispredictions. Generally speaking, deeper pipelines that require speculative scheduling will have greater misprediction penalties and will be more sensitive to this effect.
- The degree to which a program is limited by data flow dependences. If a program is primarily performance-limited by something other than data dependences, eliminating data dependences via value prediction will not

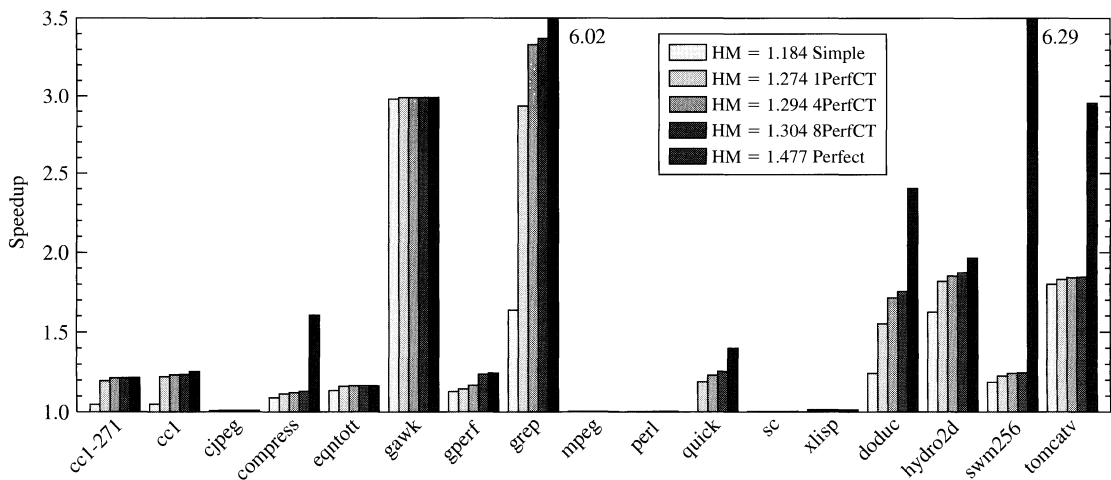
result in much benefit. For example, if a program is limited by instruction fetch, branch mispredictions, structural hazards, or memory bandwidth, it is unlikely that value prediction will help performance.

In summary, the performance effects of value prediction are not yet fully understood. What is clear is that under a large variety of instruction sets, benchmark programs, machine models, and misprediction recovery schemes, nontrivial speedup is achievable and has been reported in the literature.

As an indication of the performance potential of value prediction, some performance results for an idealized machine model are shown in Figure 10.10. This idealized machine model measures one possible data flow limit, since, for all practical purposes, parallel issue in this model is restricted only by the following three factors:

- Branch prediction accuracy, with a minimum redirect penalty of three cycles
- Fetch bandwidth (single taken branch per cycle)
- Data flow dependences
- A value misprediction penalty of one cycle

This machine model reflects idealized performance in most respects, since the misprediction penalties are very low and there are no structural hazards. However, we consider history-based value predictors only, so later studies that employed



The Simple configuration employs a straightforward last-value predictor. The 1PerfCT, 4PerfCT, and 8PerfCT configurations use perfect confidence, eliminating all mispredictions while maximizing coverage, and choosing from a value history of 1, 4, or 8 last values, respectively. The Perfect configuration eliminates all true data dependences and indicates the overall performance potential.

Figure 10.10

Value Prediction Speedup for an Idealized Machine Model.

Table 10.1

Value prediction unit configurations

Configuration	Value Prediction Table		Confidence Table	
	Direct-Mapped Entries	History Depth	Direct-Mapped Entries	Bits/Entry
Simple	4096	1	1024	2-bit up-down saturating counter
1PerfCT	4096	1	∞	Perfect
4PerfCT	4096	4/Perfect selector	∞	Perfect
8PerfCT	4096	8/Perfect selector	∞	Perfect
Perfect	∞	Perfect	∞	Perfect

computational or hybrid predictors have shown dramatically higher potential speedup. Figure 10.10 shows speedup for five different value prediction unit configurations, which are summarized in Table 10.1. Attributes that are marked *perfect* in Table 10.1 indicate behavior that is analogous to *perfect caches*; that is, a mechanism that always produces the right result is assumed. More specifically, in the 1PerfCT, 4PerfCT, and 8PerfCT configurations, we assume an *oracle confidence table (CT)* that is able to correctly identify all predictable and unpredictable register writes. Furthermore, in the 4PerfCT and 8PerfCT configurations, we assume a perfect mechanism for choosing which of the four (or eight) values stored in the value history is the correct one. Note that this is an idealized version of the last- n predictor proposed by Burtcher and Zorn [1999]. Moreover, we assume that the perfect configuration can always correctly predict a value for every register write, effectively removing all data dependences from execution. Of these configurations, the only value prediction unit configuration that we know how to build is the simple one, while the other four are merely included to measure the potential contribution of improvements to both value prediction table (VPT) and CT prediction accuracy.

The results in Figure 10.10 clearly demonstrate that even simple predictors are capable of achieving significant speedup. The difference between the simple and 1PerfCT configurations demonstrates that accuracy is vitally important, since it can increase speedup by a factor of 50% in the limit. The 4PerfCT and 8PerfCT cases show that there is marginal benefit to be gained from history-based predictors that track multiple values. Finally, the perfect configuration shows that dramatic speedups are possible for benchmarks that are limited by data flow.

10.4.6 Concluding Remarks

In summary, various schemes for speculative execution based on value prediction have been proposed. Researchers have described techniques for improving prediction accuracy and coverage and focusing predictor scope to where value predictions are

perceived to be most useful. Many implementation issues, both in predictor design as well as effective microarchitectural support for value-speculative execution have been studied. At the same time, numerous unanswered questions and unexplored issues remain. No real designs that incorporate value prediction have yet emerged; only time will tell if the demonstrated performance potential of value prediction will compensate for the additional complexity required for its effective implementation.

10.5 Summary

This chapter has explored both speculative and nonspeculative techniques for improving register data flow beyond the classical data flow limit. These techniques are based on the program characteristic of value locality, which describes the likelihood that previously seen operand values will recur in later executions of static program instructions. This property is exploited to remove computations from a program's dynamic data flow graph, potentially reducing the height of the tree and allowing a compressed execution schedule that permits instructions to execute sooner than their position in the data flow graph might indicate. Whenever this scenario occurs, a program is said to be executing beyond the data flow limit, which is a rate computed by dividing the number of instructions in the data flow graph by the height of the graph. Since the height is reduced by these techniques, the rate of execution increases beyond the data flow limit.

The nonspeculative techniques range from memoization, which is a programming technique that stores and reuses the results of side-effect free computations; to instruction reuse, which implements memoization at the instruction level by reusing previously executed instructions whenever their operands match the current instance; to block, trace, and data flow region reuse, which extend instruction reuse to larger groups of instructions based on control or data flow relationships. Such techniques share the characteristic that they are only invoked when known to be safe for correctness; safety is determined by applying a *reuse test* that guarantees correctness. In contrast, the remaining value locality-based technique that we examined—value prediction—is speculative in nature, and removes computation from the data dependence graph whenever it can correctly predict the outcome of the computation. Value prediction introduces additional microarchitectural complexity, since speculative execution, misprediction detection, and recovery mechanisms must all be provided.

None of these techniques has yet been implemented in a real processor design. While published studies indicate that dramatic performance improvement is possible, it appears that industry practitioners have found that incremental implementations of these techniques that augment existing designs do not provide enough performance improvement to merit the additional cost and complexity. Only time will tell if future microarchitectures, perhaps more amenable to adaptation of these techniques, will actually do so and reap some of the benefits described in the literature.